

Analisi di dati sui voli con Apache Spark

Progetto 1 - SABD

Ludovico Di Iorio

0336019

ludovico.diiorio@students.uniroma2.eu

1. ABSTRACT

Questo documento descrive il lavoro fatto per la realizzazione di un sistema che sfrutta il framework *Apache Spark* per fare processamento batch distribuito.

Vengono mostrate le soluzioni trovate per far fronte alla realizzazione del sistema, ed alla fine del documento verranno eseguiti degli esperimenti per valutare le prestazioni del sistema realizzato.

2. OBIETTIVO DEL PROGETTO

Il seguente progetto ha lo scopo di analizzare il dataset contenente i dati storici relativi ai voli, fornito dal Dipartimento dei trasporti degli Stati Uniti d'America. Il sistema realizzato, per perseguire tale scopo, adopererà diversi framework, tra cui:

- **Apache Spark:** framework per il data-processing distribuito.
- **Hadoop Distributed File System:** framework per realizzare un file system distribuito.
- **Apache Airflow:** framework per la definizione di pipeline, schedulazione e monitoraggio di task dentro una pipeline.
- **Apache Zeppelin:** notebook web per analisi dati.

Il sistema, tramite questi framework, dovrà eseguire due query per estrarre determinate informazioni dal dataset:

- a) **Query 1:** facendo riferimento al dataset dei voli relativi alle compagnie aeree AA (American Airlines) e DL (Delta), aggregare i dati su base mensile. Calcolare la media, il minimo ed il massimo di DEP DELAY e il CANCELLATION RATE per ciascun mese (in esame) del 2025. Il CANCELLATION RATE deve essere calcolato come la percentuale di voli cancellati sul totale dei voli osservati per ciascuna compagnia e per

ciascun mese. Nel calcolo delle statistiche relative ai ritardi (DEP DELAY, ARR DELAY), considerare esclusivamente i voli non cancellati. Il CancellationRate deve invece essere calcolato considerando tutti i voli osservati nel gruppo di aggregazione. Inoltre, considerando il valor medio di DEP DELAY e il CANCELLATION RATE aggregati su base mensile, generare due grafici che consentano di confrontare visivamente l'andamento per AA e DL.

- b) **Query 2:** Per ciascuna compagnia (OP UNIQUE CARRIER), calcolare il numero totale di voli non cancellati (CANCELLED) e non deviati (DIVERTED), il valor medio di ARR DELAY e il valor medio delle componenti di ritardo CARRIER DELAY, WEATHER DELAY, NAS DELAY, SECURITY DELAY e LATE AIRCRAFT DELAY. Si considerino solo le compagnie con almeno 500 voli non cancellati e non deviati nel periodo osservato. Calcolare quindi la classifica delle prime 10 compagnie ordinate per ARR DELAY medio decrescente. Per ciascuna delle 10 compagnie in classifica, mostrare il contributo medio delle diverse cause di ritardo. Si noti che le componenti di ritardo possono contenere valori mancanti; tali valori possono essere trattati come zero oppure ignorati, purché la scelta sia chiaramente motivata.

I risultati ottenuti poi saranno salvati su HDFS come file .csv e analizzati su un notebook Zeppelin.

3. ARCHITETTURA DEL SISTEMA

Il sistema adotta un'architettura distribuita basata su container Docker, in cui ogni componente applicativo è eseguito in un container separato e svolge un ruolo specifico all'interno del flusso di elaborazione dei dati. I

container sono organizzati in tre cluster logici che comunicano tra loro; ciascun cluster raggruppa servizi con funzionalità omogenee e si occupa di un aspetto ben definito del funzionamento complessivo del sistema (ad esempio persistenza dei dati, elaborazione con Spark, esposizione dei risultati).

Cluster A: persistenza dei dati

Questo cluster ha lo scopo di gestire la persistenza dei dati, e si affida al framework **HDFS** (Hadoop Distributed File System). Questo framework si basa su un'architettura master-slave distribuita, per cui sarà presente un container Docker dedicato al master e n container per memorizzare in maniera distribuita i file del sistema.

Cluster B: elaborazione con Spark

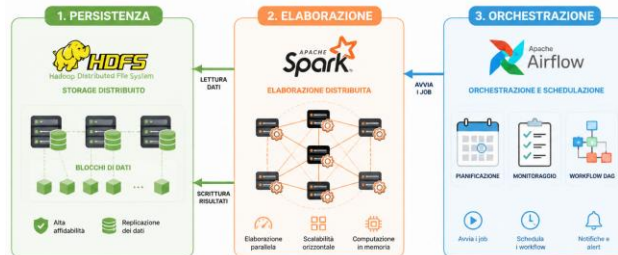
Su questo cluster di container avviene il processamento: il framework utilizzato per raggiungere tale scopo è **Apache Spark**, che si basa su un'architettura master-worker distribuita. Il cluster sarà composto da un container dedicato al master (il Resource Manager), un container su cui gira il Driver Program ed m container per i worker del cluster, su tale cluster gireranno due job Spark: uno per pre-processare i dati, quindi per rimuovere dal dataset le informazioni inutili ai fini dell'analisi, e poi un secondo job per processare i dati e ricavare le informazioni richieste dalle query.

Cluster C: orchestrazione dei task

Questo è il cluster che si occupa dell'orchestrazione del workflow complessivo e si basa sul framework **Apache Airflow**. Attraverso un *DAG*, viene dichiarato l'ordine d'esecuzione dei due task da eseguire: il primo task si occupa di pre-processare i dati del dataset, mentre il secondo task (avviato solamente quando il primo si è concluso con successo) si occupa dell'elaborazione vera e propria, quindi di generare i risultati richiesti dalle query. Airflow si basa su un'architettura distribuita, quindi è formato da diversi componenti, ognuno dei quali è stato containerizzato e ha una precisa responsabilità nel ciclo di vita del DAG:

- *Postgres*: database di metadati di Airflow, che memorizza lo stato dei DAG, dei task, delle variabili e delle esecuzioni.
- *API-server*: espone le API e supporta l'accesso al sistema
- *Scheduler*: legge i DAG, controlla dipendenze e timing, e invia i task all'executor quando devono partire.

- *Dag-processor*: ha il compito di analizzare i file DAG, interpretarli e serializzarli nel metadata database, così che il resto di Airflow possa lavorare su una rappresentazione aggiornata e ordinata dei workflow.
- *Triggerer*: gestisce i task differiti, cioè i task che non devono occupare risorse mentre aspettano un evento esterno o una condizione di completamento.



Airflow si occupa di avviare il workflow e di garantire che il task Spark, che analizza il dataset, parta solamente se il task Spark che pre-processa i dati abbia terminato il suo compito con successo. Le operazioni eseguite dai task Spark avvengono in maniera distribuita sui container del *cluster B* (sugli m nodi worker) e ognuno dei due task comunica con il cluster HDFS per recuperare il dataset originale, il dataset processato e per salvare i risultati delle query.

L'architettura del sistema è stata progettata per separare le responsabilità dei singoli componenti, facendo sì che ogni componente avesse un suo ambiente isolato (il container) per essere eseguito. Tale scelta agevola la scalabilità del sistema, infatti si possono aggiungere e rimuovere sia i DataNode di HDFS che i nodi worker di Spark molto semplicemente.

4. IMPLEMENTAZIONE

In questa sezione vengono descritti gli aspetti implementativi del sistema, con particolare attenzione alle tecnologie adottate, alla configurazione dei container e all'integrazione tra i diversi componenti dell'architettura.

Il sistema è stato sviluppato e deployato in un ambiente locale basato su Docker, utilizzando una rete virtuale condivisa tra tutti i container. Questa scelta ha permesso di semplificare la comunicazione tra i servizi grazie al DNS interno di Docker, che consente di indirizzare i componenti tramite il nome del container anziché usare gli indirizzi IP.

Implementazione Cluster A

Come già introdotto, il *cluster A* sarà composto da $n+1$ container, dove uno di questi è il NameNode e gli altri n sono i DataNode. Entrambi questi container sono stati creati utilizzando come immagine Docker di partenza *matnar/hadoop:3.3.2*, la quale consente di avviare un container per un cluster HDFS in maniera molto semplice, però, a differenza dei DataNode, il NameNode utilizza un'immagine custom (che estende *matnar/hadoop:3.3.2*), a cui è stato associato un volume Docker con dentro i file che compongono il dataset, e poi viene anche caricato uno script *.sh* per agevolare la configurazione dell'ambiente HDFS.

```
1 #!/bin/sh
2
3 hdfs namenode -format
4
5 $HADOOP_HOME/sbin/start-dfs.sh
6
7 hdfs dfs -mkdir -p /user/root
8
9 hdfs dfs -mkdir /user/root/data
10
11 cd /hadoop/dfs/data
12
13 hdfs dfs -put 202501_T_ONTIME_REPORTING.csv data/
14 hdfs dfs -put 202502_T_ONTIME_REPORTING.csv data/
15 hdfs dfs -put 202503_T_ONTIME_REPORTING.csv data/
16 hdfs dfs -put 202504_T_ONTIME_REPORTING.csv data/
17
18 hdfs dfs -chown -R airflow data/
```

(start.sh)

All'interno di questo script sono prima eseguite le operazioni di inizializzazione e avvio di HDFS, poi vengono caricati sul file system distribuito i file che compongono il dataset, ed infine vengono dati i permessi di scrittura e lettura all'utente *airflow*, perché altrimenti i task Spark avviati dalla pipeline Airflow non possono né leggere il dataset, né scrivere i risultati delle query.

I file *.csv* che compongono il dataset sono stati memorizzati separatamente poiché Spark offre la possibilità di caricare tutti i dati dei file contenuti in una directory con una sola operazione all'interno di un RDD o di un Dataframe. Dopo aver eseguito lo script, il cluster sarà pronto e accessibile.

Implementazione Cluster B

Questo cluster sarà composto da $m+1$ container, ed essi sono avviati tramite un *docker-compose.yml* file, che consente di definire le caratteristiche dei container e tramite un semplice comando di avviarli.

docker compose up --scale spark-worker=m

Ogni container del *cluster B* è costruito utilizzando un'immagine Docker custom che si basa su *apache/spark-py:v3.4.0*, ma assumeranno ruoli differenti all'interno del cluster in base alle configurazioni e ai comandi eseguiti all'avvio del container, definiti all'interno del *docker-*

compose.yml; più precisamente ci saranno due tipi di container Spark:

- container *pyspark-master*: questo è il container su quale gira il master del cluster Spark, ovvero dove si trova il Cluster Manager (*Standalone mode*);

```
spark-master:
  container_name: spark-master
  build:
    context: .
    dockerfile: ./spark-images/Dockerfile
  networks:
    - sharednet
  environment:
    SPARK_MODE: master
  command: >
    /opt/spark/bin/spark-class
    org.apache.spark.deploy.master.Master
    --host spark-master
    --port 7077
    --webui-port 8080
  ports:
    - "7077:7077"
    - "8080:8080"
```

- container *spark-worker*: ci saranno *m* container di questo tipo, e sono i container che si occupano di avviare gli executors e quindi di eseguire i task.

```
spark-worker:
  build:
    context: .
    dockerfile: ./spark-images/Dockerfile
  environment:
    SPARK_MODE: worker
    SPARK_MASTER_URL: spark://spark-master:7077
  networks:
    - sharednet
  depends_on:
    - spark-master
  command: >
    /opt/spark/bin/spark-class
    org.apache.spark.deploy.worker.Worker
    spark://spark-master:7077
```

Per quanto riguarda il *driver program*, questo componente del cluster non ha un container dedicato e ciò dovuto dalla modalità con la quale i task Spark sono lanciati: come già detto, i task Spark sono avviati da una pipeline di Airflow, la quale consente di eseguirli in modalità *cluster* o in modalità *client*. Nel sistema realizzato, i task sono eseguiti in modalità *client* e questo significa che il *driver program* gira sul nodo che lancia i task ed in questo caso è uno dei container del cluster di Airflow (*cluster C*).

Dopo aver avviato il cluster, i nodi saranno fin da subito raggiungibili e pronti per eseguire i task Spark.

Implementazione Cluster C

Il *cluster C* è il centro di coordinamento dell'intero sistema e tramite il framework *Apache Airflow* avviene l'orchestrazione dei task da eseguire. I container con le loro caratteristiche sono definiti all'interno dello stesso *docker-compose.yml* file utilizzato per tirare su il *cluster B*, e ogni container del cluster, ad eccezione di *Postgres*, è costruito

tramite un Dockerfile che prende come immagine Docker di partenza *apache/airflow:3.2.1-python3.10*, e poi ogni elemento del cluster si distingue per il comando che viene lanciato all'avvio (p.e. nel container dello scheduler il comando lanciato all'avvio è *'scheduler'*).

La pipeline viene definita all'interno del file *dag.py*, nel quale viene dichiarato un DAG Airflow, che si occupa di esplicitare i task Spark da eseguire tramite il componente *SparkSubmitOperator* e di definire la dipendenza tra i task.: in *dag.py*, il task di processamento parte quando il task di pre-processamento si conclude con successo. I task sono definiti nel seguente modo:

```
task_1 = SparkSubmitOperator(
    task_id="pre_processing",
    conn_id="spark_conn",
    application="/opt/airflow/jobs/pre-processing.py",
    conf={
        "spark.master": "spark://spark-master:7077",
        "spark.submit.deployMode": "client"
    },
    application_args=[
        config["host"],
        config["port"],
        config["directory"],
        config["RDD_Dataframe"]
    ]
)
```

(il task_2 è uguale al task_1 ma cambia solo il parametro *application*)

I task sono dichiarati all'interno del DAG e alla fine viene definita la dipendenza nel seguente modo:

```
task_1 >> task_2
```

Adesso andiamo a descrivere cosa avviene all'interno dei job Pyspark: i job sono caricati sui container del *cluster C* tramite l'immagine creata con il Dockerfile e ce ne sono due:

pre-processing.py: questo job serve per eseguire un filtraggio sul dataset e rimuovere tutte le colonne del dataset che non serviranno ai fini della computazione.

Il componente che si occupa di lanciare i job sarà lo scheduler di Airflow e sempre lo scheduler passerà degli argomenti in input al job:

- *host*: host del master HDFS.
- *port*: numero di porta del master HDFS per interagire col file system distribuito.
- *directory*: radice del percorso per trovare il dataset.
- *RDD_Dataframe*: stringa per esplicitare se il pre-processamento va fatto tramite RDD o Dataframe.
- *num_partitions*: numero di partizioni con cui salvare il dataset processato.

In base al valore dichiarato in *RDD_Dataframe*, il caricamento e il filtraggio del dataset avverranno su un RDD o su Dataframe.

All'inizio avviene il caricamento del dataset facendo interagire il job Pyspark con HDFS e poi verranno rimosse le colonne inutili del dataset, infatti rimarranno solamente ***month, op_unique_carrier, dep_delay, arr_delay, cancelled, diverted, carrier_delay, weather_delay, nas_delay, security_delay e late_aircraft_delay***.

Terminato il pre-processamento, il nuovo dataset viene salvato sempre su HDFS così che il prossimo job Pyspark possa accedervi, però il formato usato per il salvataggio dipende dal valore di *RDD_Dataframe*.

- Se *RDD_Dataframe* = "*RDD*", allora il dataset viene salvato su HDFS in formato *.csv*.
- Se *RDD_Dataframe* = "*Dataframe*", allora il dataset viene salvato su HDFS in formato *.parquet*.

L'esecuzione di questo job consente di avere un dataset con una dimensione molto ridotta, il che può rendere il processamento successivo molto più veloce.

Se *RDD_Dataframe* = "*RDD*", allora il dataset filtrato viene salvato come un file *.csv* e si avrà una riduzione della dimensione del dataset di circa il 68%.

Se *RDD_Dataframe* = "*Dataframe*", allora il dataset filtrato viene salvato come un file *.parquet* e ciò consente di avere una riduzione della dimensione del dataset migliore rispetto a quella ottenuta con il *.csv*, infatti la riduzione sarà di circa il 96%.

Una volta che il dataset ridotto è stato salvato con successo su HDFS, *pre-processing.py* termina e così Airflow potrà far partire il secondo task, quello per il processamento.

processing.py: questo è un job Pyspark che sfrutta un cluster Spark per effettuare l'analisi del dataset filtrato. Anche qui lo scheduler passa degli argomenti al task e sono:

- *host*: host del master HDFS.
- *port*: numero di porta del master HDFS per interagire col file system distribuito.
- *directory*: radice del percorso per trovare il dataset.
- *RDD_Dataframe*: stringa per esplicitare se il pre-processamento va fatto tramite RDD o Dataframe.
- *num_cores*: numero di core disponibili per l'executor.

Gli argomenti passati sono praticamente gli stessi del primo task (ad eccezione di *num_cores*), per cui se *pre-processing.py* lavora con gli RDD, allora anche *processing.py* userà gli RDD.

Dopo aver creato la *SparkSession* e avviato lo *SparkContext*, viene prelevato il dataset da HDFS ed in base al tipo di astrazione dei dati definita dall'argomento *RDD_Dataframe*, il dataset sarà preso da un *.csv* e caricato su un RDD oppure preso da un *.parquet* file e caricato su un *Dataframe*.

Quando si usa l'RDD come astrazione per i dati è necessario effettuare subito una trasformazione per far diventare una riga del *.csv* (che viene vista come una stringa) una lista di stringhe, il che rende più semplice l'elaborazione.

Quando il dataset è stato caricato con successo, viene persistito in memoria tramite il metodo *.cache()* (visto che viene utilizzato sia per la query1 che per la query2) e parte subito la prima query.

Entrambe le query sono state realizzate in due versioni: una per lavorare con RDD e una per lavorare con *Dataframe*; di conseguenza, andremo a descrivere il funzionamento di entrambe le versioni delle query:

Query 1 con RDD: vengono eseguite le seguenti trasformazioni in cascata:

- 1) *.filter()*: vengono rimossi dall'RDD tutte le righe che non riguardano i carrier AA e DL.
- 2) *.map()*: la lista di stringhe è trasformata in una coppia chiave valore in cui la chiave è data dalla coppia (*month*, *op_unique_carrier*) e il valore è un tupla composta da tutti gli altri elementi.
- 3) *.aggregateByKey()*: trasformiamo le singole tuple in una nuova tupla (*seq_func1()*) e vengono poi aggregate secondo la chiave (*comb_func1()*)
- 4) *.map()*: trasformiamo ogni tupla in una stringa che segue il formato *.csv*
- 5) *.coalesce(1)*: combina tutte le partizioni esistenti in una sola, così quando si scrive su HDFS, verrà generato una sola partizione.

Infine, viene messa l'azione *.saveAsTextFile()* per salvare i risultati della query su HDFS in formato *.csv*.

```
dataset.filter(lambda filter_carrier) \
    .map(lambda x: ((x[0], x[1]), (x[2:]))) \
    .aggregateByKey((0.0, float("inf"), float("-inf"), 0, 0), lambda seq_func1, lambda comb_func1) \
    .map(lambda tupla_finale1) \
    .coalesce(1) \
    .saveAsTextFile(hdfs_path + "output/query1.csv")
```

Query 1 con Dataframe: anche in questo caso, la prima trasformazione riguarda il filtraggio, quindi la rimozione dal dataset di tutte le righe non inerenti ai carrier AA e DL.

Il dataframe rende molto più semplice effettuare l'aggregazione e le operazioni di aggregazione; infatti, in una sola riga è possibile ottenere un dataframe con i valori richiesti dalla query.

E poi infine c'è il salvataggio dei risultati della query su HDFS in formato *.csv*.

```
filtered_df = dataset.filter(dataset.OP_UNIQUE_CARRIER.isin('AA', 'DL'))
df = filtered_df.groupBy(["MONTH", "OP_UNIQUE_CARRIER"]).agg(
    F.avg("DEP_DELAY").alias("dep_delay_mean"),
    F.min("DEP_DELAY").alias("dep_delay_min"),
    F.max("DEP_DELAY").alias("dep_delay_max"),
    (F.sum("CANCELLED") / F.count("*") * 100).alias("cancellation_rate")
)
df.write.mode("overwrite").option("header", "true").csv(hdfs_path + "output/query1.csv")
```

Una volta che la prima query è terminata e il risultato è stato salvato su HDFS, viene avviato il processamento per la seconda query.

Query 2 con RDD: vengono eseguite le seguenti trasformazioni a cascata:

- 1) *.map()*: trasformiamo la tupla di partenza in una coppia chiave-valore, dove la chiave è *op_unique_carrier*, mentre il valore è una tupla composta da tutti gli altri elementi.
- 2) *.aggregateByKey()*: aggregiamo le tuple tramite la chiave, e la tupla risultante avrà i valori cumulativi sul numero di voli, il numero di voli cancellati, numero di voli devianti, ecc..
- 3) *.filter()*: rimuoviamo dall'RDD risultante i carrier per cui il numero di voli non cancellati o non devianti è minore di 500.
- 4) *.map()*: trasformiamo ogni tupla dell'RDD nella tupla finale che contiene i valori medi per i ritardi.
- 5) *.takeOrdered()*: vengono presi le prime 10 tuple ordinate in maniera decrescente secondo *arr_delay_mean*.
- 6) *.map()*: trasformiamo ogni tupla in una stringa che segue il formato *csv*
- 6) *.coalesce(1)*: combina tutte le partizioni esistenti in una sola, così quando si scrive su HDFS, verrà generato un solo file.

Infine, viene messa l'azione *.saveAsTextFile()* per salvare i risultati della query su HDFS in formato *.csv*.

```
rdd_list = dataset.map(lambda x: ((x[1]), (x[2:]))) \
    .aggregateByKey((tuple(0.0 for _ in range(10))), lambda seq_func2, lambda comb_func2) \
    .filter(lambda x: x[1][0]-x[1][1]-x[1][2] > 500) \
    .map(lambda tupla_query2) \
    .sortBy(lambda x: x[2], ascending=False) \
    .take(10)
sc.parallelize(rdd_list).map(lambda tupla_finale2).coalesce(1) \
    .saveAsTextFile(hdfs_path + "output/query2.csv")
```

La scelta di usare *coalesce(1)* è dovuta al fatto che si conosce la dimensione dei file *.csv* finali che contengono i risultati delle query.

Query 2 con Dataframe: a partire dal *Dataframe* che contiene il dataset, viene prima eseguita la funzione

`groupBy()`, con la quale viene eseguita un'aggregazione raggruppando per il carrier (`op_unique_carrier`) e per ogni gruppo ricavato vengono calcolati i valori cumulativi che poi serviranno per calcolare i risultati finali. Poi, con il metodo `withColumn()`, per ogni gruppo vengono calcolati i valori finali, quindi il ritardo medio all'arrivo e il ritardo medio per ogni tipo di ritardo.

Infine, il Dataframe viene ordinato in maniera decrescente per `arr_delay_mean`, vengono estratte le prime dieci tuple e poi salvate su HDFS in formato `.csv`.

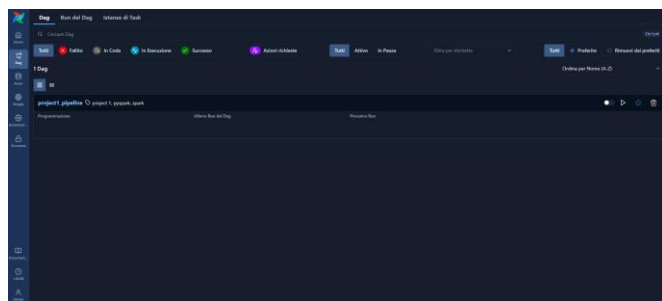
```
temp_df = dataset.groupBy("OP_UNIQUE_CARRIER").agg(
    F.count("CARRIER_ID").alias("n_flights"),
    F.sum("ARR_DELAY").alias("arr_delay_mean"),
    F.sum("CARRIER_DELAY").alias("carrier_delay_mean"),
    F.sum("WEATHER_DELAY").alias("weather_delay_mean"),
    F.sum("NAS_DELAY").alias("nas_delay_mean"),
    F.sum("SECURITY_DELAY").alias("security_delay_mean"),
    F.sum("LATE_AIRCRAFT_DELAY").alias("late_aircraft_delay_mean"),
    F.count("LATE_AIRCRAFT_DELAY").alias("late_aircraft_delay_count")
)

df = temp_df.withColumn("arr_delay_mean", F.col("arr_delay_mean") / F.col("n_flights")) \
    .withColumn("carrier_delay_mean", F.col("carrier_delay_mean") / F.col("late_aircraft_delay_count")) \
    .withColumn("weather_delay_mean", F.col("weather_delay_mean") / F.col("late_aircraft_delay_count")) \
    .withColumn("nas_delay_mean", F.col("nas_delay_mean") / F.col("late_aircraft_delay_count")) \
    .withColumn("security_delay_mean", F.col("security_delay_mean") / F.col("late_aircraft_delay_count")) \
    .withColumn("late_aircraft_delay_mean", F.col("late_aircraft_delay_mean") / F.col("late_aircraft_delay_count")) \
    .drop("late_aircraft_delay_count")

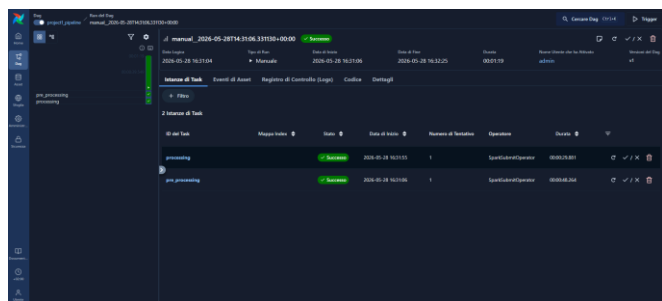
df_top10 = df.orderBy(F.col("arr_delay_mean").desc()).limit(10)
df_top10.write.mode("overwrite").option("header", "true").csv(hdfs_path + "output/query2.csv")
```

Ci sono alcune casi in cui le componenti di ritardo hanno dei valori mancanti, e per questo tali valori verranno ignorati ai fini del calcolo dei valori medi delle componenti di ritardo, poiché se venissero trattati come 0 potrebbero avere un impatto non indifferente nel valore medio finale.

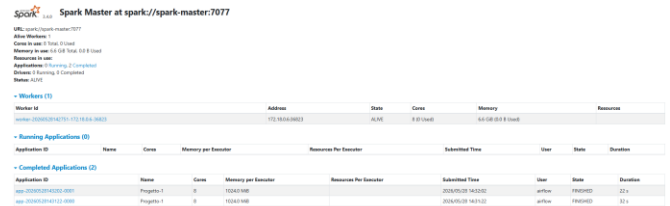
La pipeline che manda in esecuzione i task viene lanciata tramite il servizio web di Airflow, la quale sarà visualizzabile se e solo se non ci sono errori nel file `dag.py`.



La pipeline può essere triggerata e si potrà vedere lo stato e l'esito dell'esecuzione dei singoli task, sia dalla web UI di Airflow che dalla web UI di Spark.



(Airflow web UI)



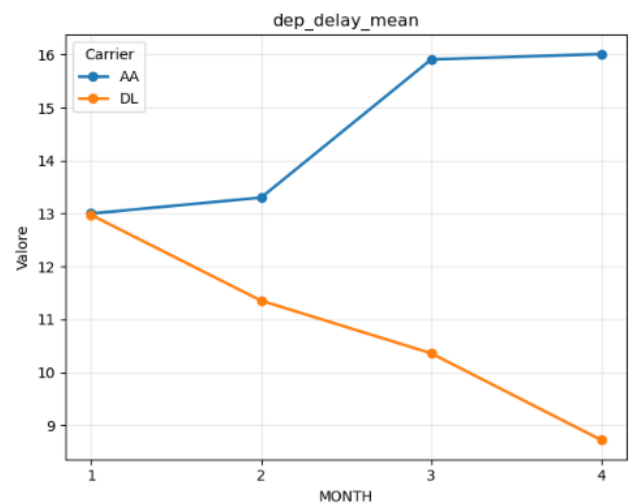
(Spark web UI)

5. RISULTATI

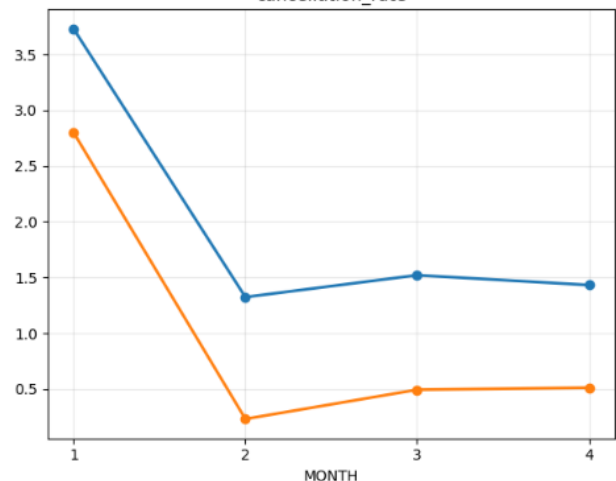
Dopo che i risultati delle query vengono salvati su HDFS, è possibile fare un'analisi dei dati e costruire dei grafici a partire da essi.

Per la generazione dei grafici viene usato il framework *Apache Zeppelin* che consente di creare un notebook ed eseguire operazioni sia di analisi che operazioni per la creazione di grafici.

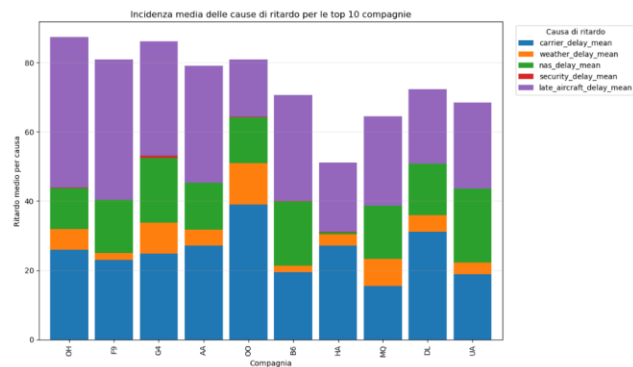
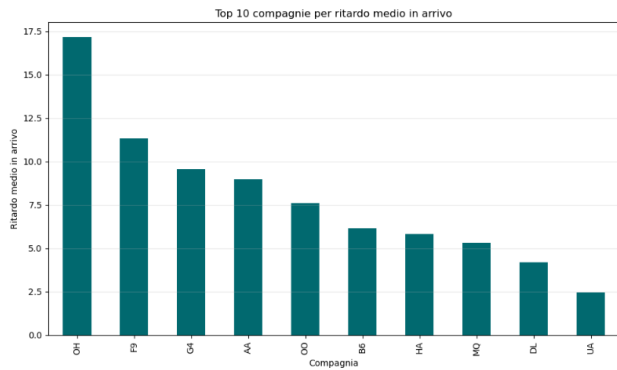
QUERY 1



cancellation_rate



QUERY 2



6. PRESTAZIONI

Il sistema è stato progettato anche col fine di poter analizzare le performance al variare della configurazione, infatti verranno fatti diversi esperimenti:

- 1) A parità di n e m , confrontiamo le performance del sistema quando lavora con RDD (e *csv*) e quando lavora con Dataframe (e *parquet*).
- 2) Al variare del grado di parallelismo, ovvero al variare del numero di core associati agli executor, confrontiamo le performance.

Le misurazioni dell'esperimento vengono raccolte dai file di log prodotti dai task *pre-processing.py* e *processing.py* piuttosto che i tempi presi dalle web UI dei framework che sono più grandi e generici poiché includono anche operazioni che non riguardano l'effettivo processamento.

Esperimento 1

Per il primo esperimento ci saranno :

- $n=1$ nodi worker per HDFS;
- 16 partizioni del dataset pre-processato;
- $m=1$ nodi worker per spark;
- ogni executor ha a disposizione 8 core.

Per l'esperimento verranno presi i tempi di esecuzione della query 1 e della query 2 su 10 run e verrà calcolato l'intervallo di confidenza del 95%:

	QUERY 1	QUERY 2
RDD	5.412 ± 0.335	7.063 ± 0.686
DATAFRAME	5.069 ± 0.242	3.985 ± 0.437

I dati raccolti mostrano che l'esecuzione delle query sfruttando i Dataframe, piuttosto che gli RDD, porta dei giovamenti, infatti il tempo per l'esecuzione della prima query, anche se di poco, è inferiore; per la seconda query si ha un guadagno notevole, il tempo d'esecuzione diminuisce di circa il 40%.

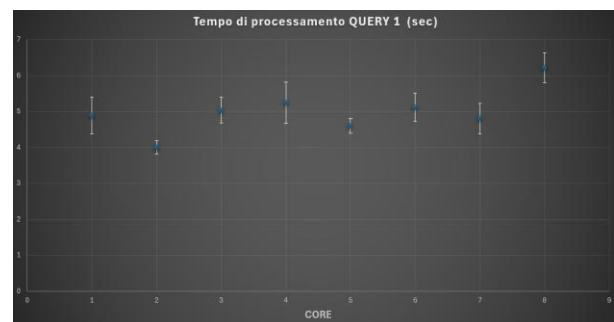
Esperimento 2

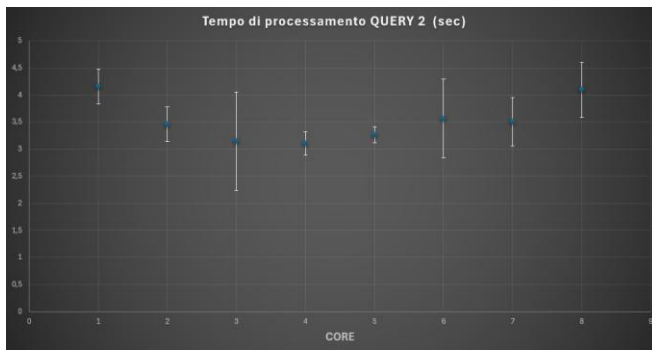
Per questo esperimento ci saranno :

- $n=1$ nodi worker per HDFS
- 16 partizioni del dataset pre-processato;
- $m=1$ nodi worker per spark;

Verrà fatto variare il numero di core (tra 1 e 8) che l'executor avrà a disposizione e verrà calcolato l'intervallo di confidenza al 95% per il tempo di processamento delle due query su 6 run.

CORE	QUERY 1	QUERY 2
1	$4,890 \pm 0,506$	$4,154 \pm 0,318$
2	$4,001 \pm 0,184$	$3,458 \pm 0,317$
3	$5,043 \pm 0,354$	$4,146 \pm 0,908$
4	$5.244 \pm 0,569$	$3,107 \pm 0,219$
5	$4,608 \pm 0,208$	$3,268 \pm 0,151$
6	$5,122 \pm 0,393$	$3,586 \pm 0,722$
7	$4,806 \pm 0,428$	$3,504 \pm 0,448$
8	$6,220 \pm 0,416$	$4,095 \pm 0,504$





I dati raccolti per la prima query non mostrano variazioni significative sulle prestazioni variando il numero di core, ad eccezioni di quando i core sono 2.

Invece, il grafico con i tempi di processamento della seconda query mostra che fino ad un certo grado di parallelismo si ha un miglioramento, e dopo il caso in cui ci sono 4 core disponibili, le prestazioni degradano progressivamente.

È interessante notare come i tempi migliori del processamento sono stati ottenuti avendo un numero di core che è un divisore del numero di partizioni, e ciò potrebbe indicare che con un numero di core che è un divisore del numero di partizioni si riesce a sfruttare meglio il parallelismo e ad avere meno overhead.

7. CONCLUSIONE

Il sistema gira localmente, per cui le risorse disponibili per l'esecuzione dei container e dei job sono limitate, e ciò ha ripercussioni sulle performance.

Se il sistema fosse effettivamente distribuito su vari host, le risorse aumenterebbero e porterebbero giovamento dal punto di vista prestazionale, riuscendo a sfruttare al meglio il parallelismo.

Infine, è giusto far notare che gli intervalli di confidenza generati nel secondo esperimento sono stati generati con solo 6 run per ogni casistica, il che non è un numero molto grande per generare intervalli di confidenza molto affidabili, però può dare una buona indicazione su quelli che sono i valori reali.